



Programación Concurrente



La **programación concurrente** es la disciplina que se encarga del estudio de las especificaciones que permiten la ejecución concurrente de un programa, así como las técnicas para resolver los problemas inherentes a ella.

Explicación

Cuando hablamos de disciplina, lo hacemos refiriéndonos al nivel más alto de abstracción.

Es decir, si hablamos de “programación concurrente” no tenemos NINGUNA IMPOSICIÓN de lenguaje de programación o arquitectura de hardware.

¿Qué es la concurrencia?

Hablamos de concurrencia cuando tenemos 2 o más procesos cuya ejecución de sus instrucciones se solape.

Es decir, cuando la primera instrucción de uno se ejecuta después de la primera instrucción de otro y antes de la última del otro.

Ejemplo:

Proceso A:

A1

A2

A3

Proceso B:

B1

B2

B3

Ejecución intercalada:

t1 → A1

t2 → B1

t3 → A2

t4 → B2

t5 → A3

t6 → B3

Aquí hay **conurrencia**, porque:

- A empezó en t1
- B empezó en t2
- Las instrucciones de A y B **se solapan en el tiempo**
- Pero **nunca se ejecutan a la vez** → no hay paralelismo

¿Qué es el paralelismo?

Aunque si dos procesos se ejecutan EXACTAMENTE al mismo tiempo, hablamos de paralelismo.

Paralelismo sería si tenemos:

t1 → CPU1 ejecuta A1 || CPU2 ejecuta B1

t2 → CPU1 ejecuta A2 || CPU2 ejecuta B2

t3 → CPU1 ejecuta A3 || CPU2 ejecuta B3

La concurrencia es más general que el paralelismo.

Podemos decir que todo sistema concurrente/paralelo tiene una posible versión secuencial. Solo habría que pensar en ejecutar todo el código con 1 solo hilo, de tal modo que imposibilitamos la posibilidad de que haya paralelismo o intercalado de ningún tipo (al haber 1 solo hilo, solo se puede estar ejecutando 1 a la vez).

La concurrencia y el hardware

Dependiendo del hardware, la concurrencia se manifiesta de forma distinta:

◆ Sistemas monoprocesador

- Solo puede ejecutarse un proceso a la vez.
- La concurrencia se logra mediante **multiprogramación** (reparto del tiempo de CPU).
- Los procesos se intercalan rápidamente, dando la ilusión de simultaneidad.

◆ Sistemas multiprocesador

- Existen varios núcleos de ejecución.
- Permiten **paralelismo real**.
- Diferentes procesos pueden ejecutarse exactamente al mismo tiempo.

◆ Sistemas distribuidos

- No existe memoria compartida entre procesos.
- La comunicación se realiza mediante **paso de mensajes**.
- Aquí la concurrencia no solo depende del tiempo, sino también de la red.

El uso de la concurrencia o no tendrá una serie de problemas y ventajas:

Beneficios de la Programación Concurrente

La ejecución concurrente puede aportar:

-  **Mayor velocidad de ejecución** (no siempre, depende del problema y de cómo se paralelice).
-  **Mejor aprovechamiento de la CPU.**
-  **Mayor capacidad de respuesta** (por ejemplo, en interfaces gráficas).
-  **Mejor escalabilidad** en sistemas con muchos núcleos.

Problemas de la Programación Concurrente

La concurrencia también introduce nuevos riesgos:

-  **Indeterminismo:**
El resultado puede depender del orden real en que se intercalen las ejecuciones.

- **✗ Condiciones de carrera** sobre recursos compartidos.
- **✗ Interbloqueos, inanición, bloqueos innecesarios.**
- **✗ Mal aprovechamiento de recursos** si la solución concurrente está mal diseñada.

Una vez que tenemos claro qué es la programación concurrente, debemos entender qué necesitamos para hacerlo bien.

Para esto, hay que saber qué significa que nuestro programa concurrente sea CORRECTO.

Corrección de Programas Concurrentes

Para que un programa concurrente sea correcto debe cumplir **dos grandes propiedades**:

Seguridad

Garantizamos que **NADA MALO** va a suceder durante la ejecución.

Esto incluye.

Exclusión mutua



Solo **una entidad concurrente** puede acceder a un recurso compartido al mismo tiempo.

Esta propiedad es imprescindible para evitar las **condiciones de carrera**, que se producen cuando el resultado de un programa depende del orden en el que se intercalan las instrucciones de los procesos.

Aunque nosotros veamos "instrucciones simples" en el alto nivel, la realidad es que cuando se traducen a instrucciones máquinas se separan en varias instrucciones, lo que genera el entrelazado.

Por ejemplo, esto:

```
contador = contador + 1
```

En lenguaje máquina son 3 instrucciones diferentes.

En realidad implica:

1. Leer `contador` desde memoria
2. Sumar 1
3. Escribir el resultado en memoria

Si dos hilos ejecutan esto a la vez sin exclusión mutua, el resultado es **impredecible**.

La exclusión mutua fuerza que este conjunto de instrucciones se ejecute de forma **atómica**.

Junto a la seguridad, un programa será correcto si aseguramos vivacidad.

Vivacidad

Ningún proceso puede quedar permanentemente esperando un recurso que otro proceso ya ha adquirido.

Incluye

Progreso

Si un proceso quiere ejecutar su sección crítica y está libre, **debe poder entrar**.

Ausencia de Interbloqueo



Ningún conjunto de procesos puede quedar bloqueado **de forma permanente** esperando recursos.

El interbloqueo aparece cuando existe:

- espera circular,
- retención mutua de recursos,
- no expropiación,
- espera indefinida.

En un interbloqueo, **todos los procesos están bloqueados y el sistema deja de progresar.**

Hilo **A** bloquea **R1** y espera **R2**
Hilo **B** bloquea **R2** y espera **R1**
→ Ambos quedan bloqueados para siempre

Ausencia de inanición

Ningún proceso puede quedar esperando **para siempre** mientras otros avanzan indefinidamente.

Bajo estas premisas se enuncia la asunción del progreso finito.

Asunción del Progreso Finito

Esta es una **hipótesis fundamental en teoría de concurrencia**:



No se hace ninguna suposición sobre la velocidad relativa o absoluta de los procesos, salvo que todas son **mayores que cero.**

Es decir:

- No se sabe cuál es más rápido.
- No se sabe cuál ejecuta antes.
- Lo único garantizado es que **todo proceso que esté listo, progresa en tiempo finito.**

Esto permite que:

- La corrección sea **independiente del hardware.**
- No dependamos del planificador del sistema operativo.
- Los razonamientos teóricos sean válidos en cualquier arquitectura.

Esta asunción nos asegura que las demoras debidas a los mecanismos de sincronización que usemos sean siempre finitas.

Vamos, que un proceso podría bloquearse y desbloquearse miles de veces, pero mientras eventualmente progrese no pasaría nada.

Esto se relaciona directamente con la propiedad de vivacidad, ya que la demora indefinida significaría inanición.

Los únicos que son una excepción a esta normal con los sistemas de tiempo real, ya que presentan una restricción superior a esta asunción.

En un sistema de tiempo real el progreso NO SOLO debe ser finito, sino que debe producirse antes de un deadline concreto (para saber más, ver los apuntes de sistemas de tiempo real).

Cuando hablamos de "progreso" nos referimos a los 2 niveles que se ven en la asignatura.

¿Qué es el progreso?

Para verificar la corrección, podemos hablar de progreso en 2 niveles:

- **Progreso global**

Existe al menos un proceso ejecutable y el sistema **no está en interbloqueo**.

El sistema **sigue avanzando**.

- **Progreso local**

Todo proceso que inicia su ejecución **eventualmente puede terminarla**.

¿Qué significa esto? En términos simples, si un hilo o proceso "progresan" es que se pueden seguir ejecutando.